

1. Playwright Test Agents

1.1 Overview

Playwright Test Agents là một tính năng AI-assisted testing được Playwright giới thiệu nhằm hỗ trợ tự động hóa quy trình xây dựng và bảo trì các bài kiểm thử End-to-End (E2E). Thay vì chỉ hỗ trợ sinh mã nguồn, Playwright chia quy trình thành ba agent chuyên biệt:

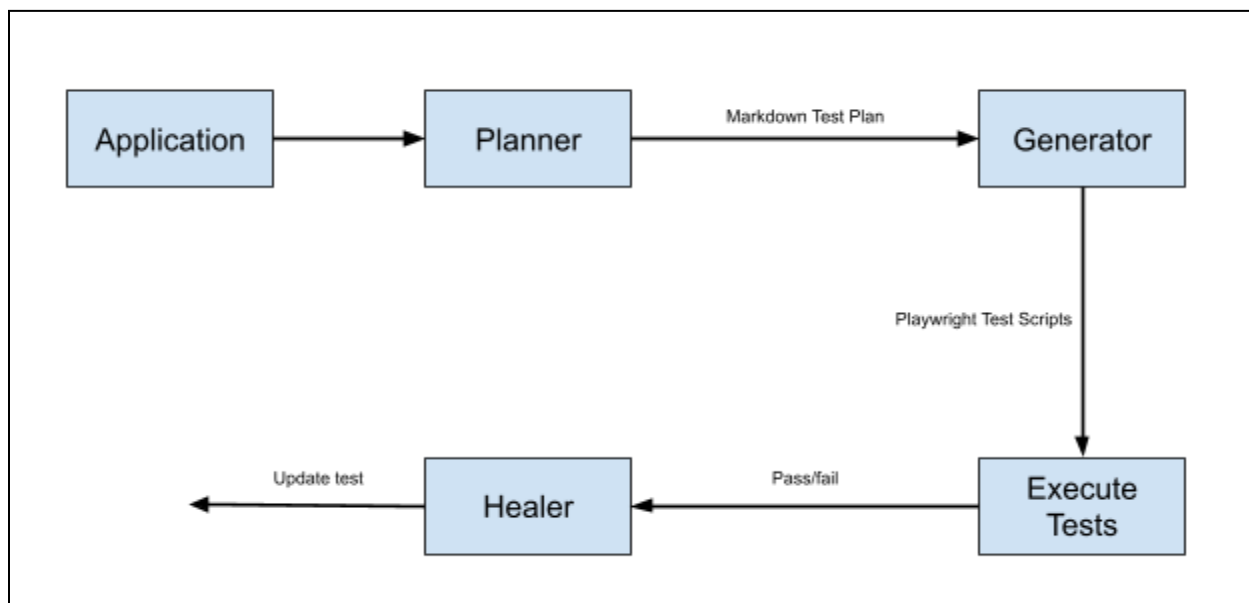
- Planner
- Generator
- Healer

Mỗi agent đảm nhận một trách nhiệm riêng, tạo thành một workflow hoàn chỉnh từ việc phân tích ứng dụng đến sinh mã kiểm thử và bảo trì các test khi giao diện thay đổi.

1.2 Playwright Agent Workflow

Playwright sử dụng mô hình nhiều agent (multi-agent workflow), trong đó mỗi agent thực hiện một giai đoạn khác nhau của quá trình kiểm thử.

Sơ đồ luồng hoạt động (Workflow):



Workflow này giúp tách biệt quá trình lập kế hoạch kiểm thử, sinh mã và bảo trì test, thay vì để một AI thực hiện toàn bộ nhiệm vụ trong một lần sinh mã.

1.3 Planner Agent

Purpose

Planner chịu trách nhiệm phân tích ứng dụng và xây dựng kế hoạch kiểm thử (test plan). Planner không sinh mã nguồn.

Đầu ra của Planner là tài liệu Markdown mô tả:

- Test Scenarios
- Test Steps
- Expected Results
- User Flows

Ví dụ:

Feature: User Login

- Scenario 1: Successful Login
- Scenario 2: Invalid Password
- Scenario 3: Locked Account

Planner có thể sử dụng:

- Seed Test
- Product Requirement Document (PRD)
- Prompt của người dùng

để hiểu ngữ cảnh trước khi tạo test plan.

1.4 Generator Agent

Purpose

Generator chuyển đổi test plan được tạo bởi Planner thành các bài kiểm thử Playwright có thể thực thi.

- **Input:** Markdown Test Plan
- **Output:** Playwright Test Files

Ví dụ: specs/login.md → tests/login.spec.ts

Khác với các công cụ chỉ sinh mã từ prompt, Generator kiểm tra trực tiếp các selector và assertion trên ứng dụng đang chạy trước khi hoàn thành test script. Điều này giúp giảm các locator không hợp lệ và tăng khả năng thực thi của test được sinh ra.

1.5 Healer Agent

Purpose

Healer được thiết kế để giảm chi phí bảo trì bộ kiểm thử.

Khi test thất bại, Healer sẽ:

- Phát lại các bước gây lỗi;
- Kiểm tra giao diện hiện tại;

- Tìm phần tử tương đương;
- Đề xuất hoặc áp dụng bản vá (ví dụ cập nhật locator hoặc điều chỉnh thời gian chờ);
- Chạy lại test để xác minh kết quả.

Nếu Healer xác định nguyên nhân đến từ lỗi của ứng dụng thay vì lỗi của test, nó có thể dừng quá trình sửa và đánh dấu đây là một lỗi chức năng cần được xử lý bởi nhóm phát triển.

1.6 Agent Artifacts

Playwright định nghĩa rõ các artifact được tạo ra trong workflow.

Artifact	Description
specs/*.md	Human-readable test plans
tests/*.spec.ts	Generated Playwright test scripts
seed.spec.ts	Seed test used to initialize application state
.github/	Agent definitions

Việc tách riêng các artifact giúp quá trình kiểm thử trở nên minh bạch và dễ kiểm soát hơn so với việc chỉ tạo trực tiếp mã kiểm thử.

1.7 Seed Test

Seed Test là một bài kiểm thử được chuẩn bị trước nhằm đưa hệ thống vào trạng thái thích hợp trước khi các agent bắt đầu hoạt động.

Ví dụ:

- Đăng nhập hệ thống;
- Tạo dữ liệu mẫu;
- Mở trang cần kiểm thử;
- Thiết lập các fixture.

Planner sử dụng Seed Test để khởi tạo môi trường, sau đó dựa trên trạng thái này để khám phá các luồng chức năng của ứng dụng. Điều này giúp các test được sinh ra ổn định hơn, đặc biệt đối với các tính năng yêu cầu xác thực hoặc dữ liệu ban đầu.

1.8 Advantages

Playwright Test Agents mang lại một số lợi ích:

- Giảm thời gian xây dựng test automation.
- Phân tách rõ ràng giữa lập kế hoạch, sinh mã và bảo trì test.
- Sinh test dựa trên ứng dụng đang chạy thay vì chỉ dựa trên prompt.
- Tự động xử lý các thay đổi nhỏ của giao diện thông qua Healer.
- Dễ tích hợp vào quy trình AI-assisted testing.

1.9 Limitations

Mặc dù hỗ trợ mạnh mẽ việc tự động hóa, Playwright Test Agents vẫn tồn tại một số hạn chế:

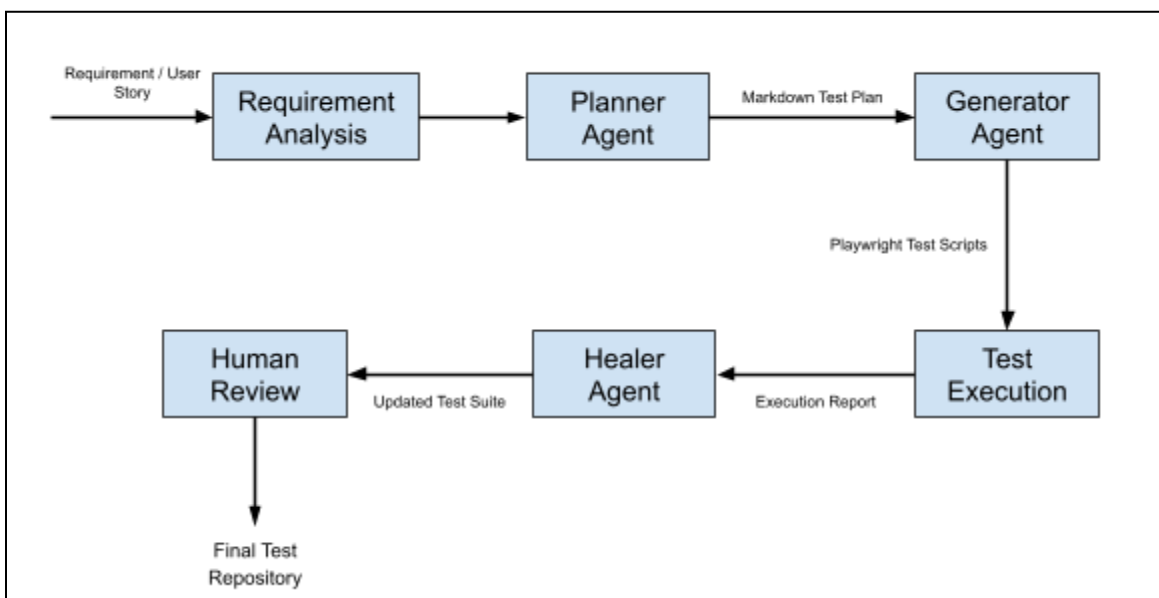
- Cần một ứng dụng đang chạy để Planner và Generator có thể khám phá giao diện.
- Chất lượng test phụ thuộc vào prompt, tài liệu đầu vào và Seed Test.
- AI có thể tạo assertion chưa đủ mạnh hoặc chưa bao phủ đầy đủ business rule, do đó vẫn cần sự xem xét của con người.
- Healer phù hợp với các thay đổi nhỏ (locator, wait, dữ liệu) nhưng không thể thay thế việc cập nhật test khi luồng nghiệp vụ thay đổi đáng kể.

2. Proposed AI-Assisted Test Generation Workflow Using Playwright Test Agents

2.1 Workflow Overview

Để tận dụng khả năng của Playwright Test Agents, đồ án đề xuất một quy trình AI-assisted test generation gồm bảy giai đoạn. Thay vì sử dụng AI chỉ để sinh mã kiểm thử, workflow này phân tách rõ trách nhiệm giữa việc lập kế hoạch kiểm thử, sinh mã, thực thi và bảo trì bộ kiểm thử.

Sơ đồ chi tiết hệ thống quy trình đề xuất:



Trong workflow này, Playwright Agent được sử dụng ở ba giai đoạn chính: Planner, Generator và Healer, còn các bước phân tích yêu cầu, thực thi và đánh giá vấn đề do quy trình kiểm thử và kỹ sư QA kiểm soát. Cách phân chia này cũng phù hợp với thiết kế chính thức của Playwright Test Agents.

2.2 Workflow Description

Step 1 - Requirement Analysis

Objective: Thu thập và phân tích yêu cầu kiểm thử.

- **Input:** User Story, Product Requirement Document (PRD), Acceptance Criteria, Existing documentation
- **Output:** Một tập yêu cầu chức năng được chuẩn hóa để làm đầu vào cho Planner Agent.

Step 2 - Test Planning

Objective: Xây dựng kế hoạch kiểm thử.

Playwright Agent: Planner

Planner thực hiện:

- Khám phá ứng dụng đang chạy;
- Sử dụng Seed Test để thiết lập trạng thái ban đầu nếu cần;
- Phân tích các luồng chức năng;
- Tạo tài liệu Markdown mô tả các kịch bản kiểm thử.
- **Input:** Requirement, Running application, Seed Test, (Optional) PRD
- **Output:** specs/ (login.md, register.md, checkout.md)

Ví dụ: Scenario 1: Valid Login, Scenario 2: Wrong Password, Scenario 3: Locked Account

Planner chỉ tạo test plan, không sinh mã kiểm thử. Điều này giúp tester xem xét và chỉnh sửa kế hoạch trước khi chuyển sang bước tiếp theo.

Step 3 - Test Generation

Objective: Sinh Playwright test scripts.

Playwright Agent: Generator

Generator đọc Markdown Test Plan do Planner tạo và chuyển thành các Playwright test có thể thực thi.

Trong quá trình sinh mã, Generator:

- Kiểm tra selector trên ứng dụng đang chạy;
- Sử dụng locator phù hợp (ví dụ getByRole, getByLabel);
- Thêm assertion và các bước kiểm thử tương ứng.
- **Input:** specs/login.md
- **Output:** tests/login.spec.ts

Generator giúp giảm đáng kể thời gian viết test thủ công và tạo ra bộ kiểm thử có cấu trúc thống nhất.

Step 4 - Test Execution

Objective: Thực thi bộ kiểm thử.

Playwright Agent: Không sử dụng Playwright Agent

Ở bước này, Playwright Test Runner chịu trách nhiệm:

- Chạy các test;
- Tạo HTML Report;
- Thu thập Trace;
- Lưu Screenshot và Video khi test thất bại.
- **Output:** HTML Report, Trace, Screenshots, Videos

Đây là nguồn dữ liệu đầu vào cho bước Healer.

Step 5 - Test Maintenance

Objective: Tự động sửa các bài kiểm thử bị lỗi.

Playwright Agent: Healer

If bài kiểm thử thất bại, Healer sẽ:

- Chạy lại các bước gây lỗi;
- Kiểm tra giao diện hiện tại;
- Tìm locator hoặc luồng tương đương;
- Đề xuất hoặc áp dụng bản vá;
- Chạy lại test để xác minh kết quả.

Ví dụ:

Old Locator: `getByText("Login")` → New Locator: `getByRole("button", { name: "Sign In" })`

Nếu nguyên nhân đến từ lỗi chức năng của hệ thống thay vì lỗi của test, Healer sẽ dừng quá trình sửa và đánh dấu đây là lỗi ứng dụng thay vì che giấu bằng cách sửa test.

Step 6 - Human Review

Objective: Đánh giá chất lượng test được sinh ra.

Playwright Agent: Không sử dụng Playwright Agent

Tester sẽ kiểm tra:

- Business rules;
- Assertion;
- Duplicate tests;
- Maintainability;
- Readability.

Đây là bước cuối cùng trước khi đưa test vào repository chính.

2.3 Role of Each Playwright Agent

Workflow Step	Agent	Responsibility
Requirement Analysis	None	Analyze functional requirements
Test Planning	Planner	Explore application and generate Markdown test plan
Test Generation	Generator	Convert test plan into executable Playwright tests
Test Execution	None	Execute Playwright test suite and collect reports
Test Maintenance	Healer	Repair failing tests and rerun execution
Human Review	None	Validate generated tests before integration

3. So sánh giữa Test viết thủ công và Test do AI sinh ra

3.1 Thời gian phát triển

Một trong những ưu điểm nổi bật nhất của AI là khả năng rút ngắn đáng kể thời gian xây dựng test automation. Trong phương pháp truyền thống, kỹ sư kiểm thử phải:

- Đọc tài liệu yêu cầu;
- Phân tích giao diện;
- Xác định locator;
- Viết Playwright script;
- Bổ sung assertion;
- Chạy thử và sửa lỗi.

Ngược lại, AI có thể tạo một Playwright test script chỉ trong vài giây từ mô tả bằng ngôn ngữ tự nhiên hoặc tài liệu yêu cầu. Điều này giúp giảm đáng kể thời gian viết mã lặp lại và tăng tốc quá trình phát triển bộ kiểm thử.

3.2 Độ bao phủ kiểm thử (Test Coverage)

Các bài kiểm thử được viết thủ công thường có độ bao phủ nghiệp vụ cao hơn do kỹ sư kiểm thử chủ động xây dựng:

- Positive Test
- Negative Test
- Boundary Value
- Edge Case
- Security Test
- Business Rule Validation

Trong khi đó, AI thường ưu tiên sinh các kịch bản theo luồng sử dụng chính (happy path). Nếu không được cung cấp đầy đủ ngữ cảnh hoặc hướng dẫn cụ thể, AI có thể bỏ sót các trường hợp biên hoặc các quy tắc nghiệp vụ quan trọng.

3.3 Chất lượng bài kiểm thử

Các bài kiểm thử viết thủ công phụ thuộc nhiều vào kinh nghiệm của người viết. Ngược lại, test do AI sinh ra thường có một số ưu điểm:

- Mã nguồn có cấu trúc thống nhất;
- Sử dụng locator hiện đại của Playwright;
- Giảm boilerplate code;
- Dễ đọc và nhất quán.

Tuy nhiên, nhiều bài kiểm thử do AI sinh ra chỉ xác minh các kết quả bề mặt như:

- URL thay đổi;
- Trang được tải thành công;
- Thông báo thành công xuất hiện.

AI thường chưa kiểm tra đầy đủ các tác động nghiệp vụ như:

- Dữ liệu đã được lưu trong cơ sở dữ liệu;
- API đã trả về đúng kết quả;
- Quyền truy cập được cập nhật chính xác.

3.4 Khả năng bảo trì

Một trong những khó khăn lớn nhất của Playwright là việc cập nhật locator khi giao diện thay đổi. Các công cụ AI hiện đại đã bắt đầu tích hợp cơ chế Self-Healing, cho phép:

- Tự động phát hiện locator mới;
- Sửa các selector bị lỗi;
- Đề xuất thay đổi phù hợp;
- Giảm chi phí bảo trì bộ kiểm thử.

3.5 Độ tin cậy

Mặc dù AI có thể sinh ra các Playwright test có thể thực thi, nhưng độ tin cậy của chúng vẫn phụ thuộc vào quá trình đánh giá của con người. Một nghiên cứu thực nghiệm công bố năm 2026 cho thấy khi cùng một AI vừa sinh mã nguồn vừa sinh bài kiểm thử, các bài kiểm thử có xu hướng xác nhận chính những giả định sai mà AI đã tạo ra trong mã nguồn. Hiện tượng này làm giảm khả năng phát hiện lỗi thực tế của bộ kiểm thử và cho thấy AI không nên là nguồn đánh giá duy nhất về chất lượng phần mềm.

3.6 Bảng so sánh

Tiêu chí	Test viết thủ công	Test do AI sinh
Tốc độ phát triển	Chậm	Nhanh
Năng suất	Thấp hơn	Cao hơn
Hiểu nghiệp vụ	Cao	Phụ thuộc ngữ cảnh
Bao phủ Edge Case	Tốt	Có thể bỏ sót
Tính nhất quán	Phụ thuộc người viết	Cao
Bảo trì	Thủ công	Có thể hỗ trợ Self-Healing
Cần Human Review	Có	Có
Nguy cơ Assertion yếu	Thấp	Cao

4. Ưu điểm của AI-Assisted Automation Testing

4.1 Rút ngắn thời gian tạo test

AI có thể nhanh chóng sinh:

- test scenario;
- Playwright test script;
- Page Object Model;
- assertion;
- fixture.

Nhờ đó, kỹ sư kiểm thử tập trung nhiều hơn vào việc đánh giá chất lượng thay vì viết mã lặp lại.

4.2 Tăng năng suất

AI hỗ trợ tự động hóa nhiều công việc lặp lại như:

- Tìm locator;
- Sinh boilerplate code;
- Tạo test regression;
- Sinh assertion cơ bản.

Điều này giúp tăng số lượng bài kiểm thử được tạo trong cùng một khoảng thời gian.

4.3 Giảm chi phí bảo trì

Các hệ thống AI hiện đại có thể hỗ trợ:

- Cập nhật locator;
- Sửa lỗi selector;
- Giảm flaky test do thay đổi giao diện.

Điều này đặc biệt hữu ích đối với các dự án có giao diện thay đổi thường xuyên.

4.4 Hỗ trợ Continuous Testing

Do AI có thể tạo và cập nhật Playwright test nhanh chóng, các tính năng mới có thể được bổ sung vào bộ regression test sớm hơn, từ đó cải thiện quy trình CI/CD và rút ngắn vòng phản hồi.

4.5 Hỗ trợ người mới

Đối với các lập trình viên hoặc QA chưa có nhiều kinh nghiệm với Playwright, AI giúp tạo nhanh bộ khung của bài kiểm thử, từ đó giảm thời gian học và triển khai ban đầu.

5. Hạn chế của AI-Assisted Automation Testing

5.1 Chưa hiểu đầy đủ nghiệp vụ

LLM có khả năng hiểu cấu trúc giao diện và mã nguồn tốt hơn so với việc hiểu các quy tắc nghiệp vụ phức tạp. Do đó, AI có thể:

- Bỏ sót yêu cầu ngầm;
- Hiểu sai business rule;
- Bỏ qua các quy định đặc thù của hệ thống.

5.2 Assertion còn đơn giản

Một hạn chế phổ biến là AI chỉ xác minh các kết quả dễ quan sát như: URL, tiêu đề trang, thông báo thành công. Trong khi đó, các kiểm tra về tính đúng đắn của dữ liệu, quyền truy cập hoặc trạng thái hệ thống thường vẫn cần được bổ sung bởi kỹ sư kiểm thử.

5.3 Phụ thuộc vào Prompt và ngữ cảnh

Chất lượng của bài kiểm thử phụ thuộc nhiều vào: prompt, tài liệu yêu cầu, tài liệu nghiệp vụ, thông tin ngữ cảnh được cung cấp. Prompt không đầy đủ thường dẫn đến các bộ kiểm thử thiếu hoặc không chính xác.

5.4 Nguy cơ lan truyền lỗi (Error Propagation)

Nếu cùng một AI vừa sinh mã nguồn vừa sinh bài kiểm thử, các bài kiểm thử có thể vô tình xác nhận các lỗi do chính AI tạo ra thay vì phát hiện chúng. Điều này làm giảm tính độc lập của bài kiểm thử và ảnh hưởng đến khả năng phát hiện lỗi của hệ thống.

5.5 Vẫn cần Human Review

Các tài liệu và nghiên cứu gần đây đều thống nhất rằng AI nên được sử dụng để sinh test, bảo trì locator, phân tích kết quả chạy test. Trong khi đó, kỹ sư kiểm thử vẫn cần:

- xác nhận business rule;
- đánh giá assertion;
- xem xét chất lượng test;
- quyết định liệu lỗi phát hiện được là lỗi của ứng dụng hay lỗi của bài kiểm thử.